

VPS Security Stack

Installazioni e Comandi OWASP Pipeline

v1.4 — Completo con guida config.env

Server: ubuntu@ta-server-openvas (51.75.196.150) — Ubuntu 23.04

Data: Maggio 2026 | Versione 1.4

1. Strumenti Operativi

Strumento	Versione	Categoria	Installazione
Nikto	2.1.5	DAST Web	apt
Semgrep	1.161.0	SAST	Docker
OWASP Dependency-Check	latest	SCA	Docker
GitLeaks	latest	Secrets	Docker
ZAP by Checkmarx	2.17.0	DAST Web	Docker (daemon mode)
DVWA	latest	Target test	Docker
OWASP ASTF	1.0-beta	DAST API	Maven/Java 21
WuppieFuzz	1.2.0	API Fuzzer	cargo
Falco	0.43.1	Runtime Security	apt + kmod
owasp-scan.sh	1.0	Pipeline script	Script bash

2. Prerequisiti

2.1 Docker

```
sudo apt update && sudo apt install -y docker.io
sudo systemctl enable --now docker
sudo usermod -aG docker $USER && newgrp docker
docker --version
```

2.2 Java 21 (richiesto per ASTF)

```
sudo apt install -y openjdk-21-jdk maven
sudo update-alternatives --config java # seleziona java-21
echo 'export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64' >> ~/.bashrc
echo 'export PATH=$JAVA_HOME/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
java -version # deve mostrare 21.x
```

2.3 Dipendenze SSL per Nikto

```
sudo apt install -y libnet-ssleay-perl
```

3. Nikto

3.1 Installazione

```
sudo apt update && sudo apt install -y nikto
nikto -Version
```

3.2 Comandi di Test

Scan base

```
nikto -h http://TARGET_IP:PORT
```

Scan su DVWA

```
nikto -h http://51.75.196.150:8081
```

Output HTML

```
nikto -h http://TARGET_IP:PORT -Format htm -output /tmp/nikto-report.html
```

Scan HTTPS

```
nikto -h https://TARGET_IP -ssl
```

Scan completo tutti i plugin

```
nikto -h http://TARGET_IP:PORT -C all
```

4. Semgrep

4.1 Installazione

```
docker pull semgrep/semgrep
echo 'alias semgrep="docker run --rm -v \"\${PWD}:/src\" semgrep/semgrep semgrep"' >> ~/.bashrc
source ~/.bashrc
semgrep --version
```

4.2 .semgreppignore

```
cat > ~/.semgreppignore << 'EOF'
.local/
go/pkg/mod/
node_modules/
vendor/
site-packages/
```

```
EOF
```

4.3 Comandi di Test

OWASP Top 10

```
cd /percorso/app  
semgrep --config=p/owasp-top-ten .
```

Secrets

```
semgrep --config=p/secrets .
```

Output SARIF

```
semgrep --config=p/owasp-top-ten --sarif --output report.sarif .
```

Test rapido

```
echo 'import subprocess; subprocess.run(input(), shell=True)' > /tmp/test.py  
cd /tmp && semgrep --config=p/python .
```

5. OWASP Dependency-Check

5.1 Installazione

```
docker pull owasp/dependency-check  
mkdir -p ~/.dependency-check-data  
cat >> ~/.bashrc << 'EOF'  
alias dependency-check='docker run --rm \  
-v "${PWD}:/src" \  
-v "${HOME}/.dependency-check-data:/usr/share/dependency-check/data" \  
owasp/dependency-check'  
EOF  
source ~/.bashrc
```

5.2 Comandi di Test

Prima esecuzione

```
nohup dependency-check \  
--scan /src --format HTML --out /src/dc-report \  
--project "nome" --nvdApiKey ef5e1703-c84b-4df3-902b-15926c8d7ff6 \  
--nvdApiDelay 6000 > ~/dc.log 2>&1 &  
tail -f ~/dc.log
```

Visualizzare report

```
cd ~/dc-report && python3 -m http.server 9000  
# Browser: http://51.75.196.150:9000/dependency-check-report.html
```

6. GitLeaks

6.1 Installazione

```
docker pull zricethezav/gitleaks  
echo 'alias gitleaks="docker run --rm -v "${PWD}:/repo" zricethezav/gitleaks"  
>> ~/.bashrc  
source ~/.bashrc
```

6.2 Comandi di Test

Storia completa commit

```
cd /percorso/repo
gitleaks detect --source /repo --verbose --log-opts="--all"
```

File correnti

```
gitleaks detect --source /repo --verbose
```

Output SARIF

```
gitleaks detect --source /repo --report-format sarif --report-path
/repo/gitleaks.sarif --verbose
```

7. DVWA — Target di Test

7.1 Avvio

```
docker run --rm -d -p 8081:80 vulnerables/web-dvwa
curl -s -o /dev/null -w "%{http_code}" http://localhost:8081 # atteso: 302
```

7.2 Configurazione

Aprire: <http://51.75.196.150:8081/setup.php> → Create/Reset Database

```
Username: admin
Password: password
```

7.3 Gestione

```
docker stop $(docker ps -q --filter ancestor=vulnerables/web-dvwa)
docker restart $(docker ps -q --filter ancestor=vulnerables/web-dvwa)
```

8. ZAP by Checkmarx

i INFO ZAP deve girare in modalità daemon per esporre l'API REST usata da owasp-scan.sh. La modalità webswing non è compatibile con lo script.

8.1 Avvio daemon (per script)

```
docker run -d --name zap -p 8080:8080 \
  ghcr.io/zaproxy/zaproxy:stable zap.sh -daemon \
  -host 0.0.0.0 -port 8080 \
  -config api.addrs.addr.name=.* \
  -config api.addrs.addr.regex=true \
  -config api.key=dallaradallara20262026

# Verifica
curl -s
"http://localhost:8080/JSON/core/view/version/?apikey=dallaradallara20262026"
# Output atteso: {"version":"2.17.0"}
```

8.2 Avvio con GUI + API

```
docker run -d --name zap -p 8080:8080 \
  ghcr.io/zaproxy/zaproxy:stable zap-webswing.sh \
```

```
-host 0.0.0.0 -port 8080 \  
-config api.addrs.addr.name=.* \  
-config api.addrs.addr.regex=true \  
-config api.key=dallaradallara20262026  
# GUI: http://51.75.196.150:8080
```

8.3 Scan via API

```
curl  
"http://localhost:8080/JSON/spider/action/scan/?apikey=dallaradallara20262026&url=TARGET_URL"  
curl  
"http://localhost:8080/JSON/ascan/action/scan/?apikey=dallaradallara20262026&url=TARGET_URL&recurse=true"  
curl  
"http://localhost:8080/JSON/alert/view/alerts/?apikey=dallaradallara20262026&baseurl=TARGET_URL" | python3 -m json.tool
```

9. OWASP ASTF

9.1 Installazione

```
git clone https://github.com/OWASP/www-project-api-security-testing-framework.git  
cd www-project-api-security-testing-framework  
mvn clean package -DskipTests  
echo 'alias astf="java -jar ~/www-project-api-security-testing-framework/target/api-security-testing-framework-1.0-SNAPSHOT.jar"' >> ~/.bashrc  
source ~/.bashrc
```

ATTENZIONE Il JAR della release GitHub è corrotto — usare sempre la build Maven dal sorgente.

9.2 Comandi di Test

Scan base

```
java -jar ~/www-project-api-security-testing-framework/target/api-security-testing-framework-1.0-SNAPSHOT.jar \  
-u http://TARGET_URL -f HTML -o ~/astf/report.html --timeout 10 -t 8 -v
```

Con Bearer token

```
java -jar ~/www-project-api-security-testing-framework/target/api-security-testing-framework-1.0-SNAPSHOT.jar \  
-u http://TARGET_URL --token "Bearer TOKEN" -f HTML -o ~/astf/report.html --timeout 10 -t 8
```

Estrai sommario report

```
python3 -c "from html.parser import HTMLParser; import re  
class P(HTMLParser):  
    def __init__(self): super().__init__(); self.f={}  
    def handle_data(self,d):  
        t=d.strip()  
        if re.match(r'ASTF-[A-Z0-9]+-\d+',t): self.f[t]=self.f.get(t,0)+1  
p=P(); p.feed(open('/home/ubuntu/astf/report.html','r',errors='ignore').read())  
[print(f'{c:6d} {t}') for t,c in sorted(p.f.items(),key=lambda x:-x[1])]"
```

10. WuppieFuzz

10.1 Installazione

```
curl --proto '=https' --tlsv1.2 -LsSf \
  https://github.com/TNO-S3/WuppieFuzz/releases/download/v1.2.0/wuppiefuzz-
installer.sh | sh
echo 'source $HOME/.cargo/env' >> ~/.bashrc && source ~/.bashrc
wuppiefuzz version
```

10.2 Fix spec OpenAPI

```
python3 -c "import json
with open('openapi.json') as f: spec=json.load(f)
spec['servers']=[{'url':'http://TARGET_URL'}]
with open('openapi-fixed.json','w') as f: json.dump(spec,f,indent=2)
print('Done')"
```

10.3 Comandi di Test

Scan black box

```
source $HOME/.cargo/env
wuppiefuzz fuzz --timeout 120 --crash-criterion only5xx openapi-fixed.json
```

Con autenticazione Bearer

```
cat > auth.yaml << 'EOF'
mode: bearer
configuration:
  url: http://TARGET_URL/login
  username: admin
  password: password
EOF
wuppiefuzz fuzz --authentication auth.yaml --timeout 300 openapi-fixed.json
```

Analisi crash

```
ls crashes/ | wc -l
cat crashes/$(ls crashes/ | head -1)
```

11. Falco — Runtime Security

i INFO Falco monitora tutti i container e il sistema host in tempo reale usando syscall a livello kernel.

11.1 Installazione

```
curl -fsSL https://falco.org/repo/falcosecurity-packages.asc | \
  sudo gpg --dearmor -o /usr/share/keyrings/falco-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/falco-archive-keyring.gpg] \
  https://download.falco.org/packages/deb stable main" | \
  sudo tee /etc/apt/sources.list.d/falcosecurity.list
sudo apt update && sudo apt install -y falco
```

11.2 Configurazione su KVM

● ATTENZIONE Su VM KVM il BPF filesystem non è montato di default.

```
sudo mount -t bpf bpf /sys/fs/bpf
echo 'bpf /sys/fs/bpf bpf defaults 0 0' | sudo tee -a /etc/fstab
sudo falcoctl driver install --type kmod
sudo insmod /root/.falco/9.1.0+driver/x86_64/falco_ubuntu-generic_6.2.0-39-
generic_40.ko
```

11.3 Avvio e monitoring

```
sudo falco -o "engine.kind=kmod" > /var/log/falco-alerts.log 2>&1 &  
sudo tail -f /var/log/syslog | grep  
'falco:.*Warning\\|falco:.*Notice\\|falco:.*Critical'
```

11.4 Test alert

Lettura file shadow

```
sudo cat /etc/shadow > /dev/null  
# Alert atteso: Warning Sensitive file opened for reading
```

Shell in container

```
docker exec -it CONTAINER_ID bash  
# Alert atteso: Notice A shell was spawned in a container
```

12. owasp-scan.sh — Pipeline Script

i INFO Script bash che esegue automaticamente tutta la catena OWASP. Modifica solo config.env — lo script non va mai toccato.

12.1 Struttura

```
~/owasp-scan/  
├─ owasp-scan.sh      ← script principale (non modificare)  
├─ config.env         ← UNICO file da configurare  
├─ reports/           ← report generati automaticamente  
│   └─ TIMESTAMP/  
│       ├── index.html ← report unificato  
│       ├── nikto-report.html  
│       ├── zap-alerts.json  
│       ├── astf-report.html  
│       └─ wuppiefuzz/crashes/
```

12.2 Installazione

```
mkdir -p ~/owasp-scan && cd ~/owasp-scan  
cp config.env.example config.env  
chmod +x owasp-scan.sh
```

12.3 Utilizzo

Pipeline completa

```
cd ~/owasp-scan  
./owasp-scan.sh
```

Solo DAST — senza codice sorgente

```
./owasp-scan.sh --skip-sast --skip-secrets --skip-sca --skip-api --skip-openvas
```

Uno strumento specifico

```
./owasp-scan.sh --only nikto  
./owasp-scan.sh --only zap  
./owasp-scan.sh --only astf
```

In background con nohup

```
nohup ./owasp-scan.sh > ~/owasp-scan/pipeline.log 2>&1 &  
tail -f ~/owasp-scan/pipeline.log
```

12.4 Prerequisiti

```
# ZAP deve essere in modalità daemon
curl -s
"http://localhost:8080/JSON/core/view/version/?apikey=dallaradallara20262026"
# Deve rispondere: {"version":"2.17.0"}

# Se è in webswing riavvialo
docker stop zap && docker rm zap
docker run -d --name zap -p 8080:8080 \
  ghcr.io/zaproxy/zaproxy:stable zap.sh -daemon \
  -host 0.0.0.0 -port 8080 \
  -config api.addrs.addr.name=.* \
  -config api.addrs.addr.regex=true \
  -config api.key=dallaradallara20262026
```

12.5 Visualizzare report

```
LATEST=$(ls -td ~/owasp-scan/reports/*/ | head -1)
cd $LATEST && python3 -m http.server 9999
# Browser: http://51.75.196.150:9999
```

12.6 Risultati test su DVWA (11/05/2026)

Strumento	Findings	Severity	Durata
Nikto	21	Info/Low — header mancanti, directory listing	7s
ZAP	276	61 Medium, 88 Low, 127 Info	~4s
TOTALE	297	0 High — scan non autenticato	11s

 **TIP** Con scan autenticato i finding High salgono — SQLi, XSS e Command Injection sono presenti in tutte le sezioni di DVWA.

13. Processi Lunghi — nohup e screen

13.1 nohup

```
nohup COMANDO > ~/output.log 2>&1 &
echo "PID: $!"
tail -f ~/output.log
kill PID # per fermarlo
```

13.2 Monitoraggio risorse

```
watch -n 5 'top -bn1 | head -8 && free -h'
docker ps
df -h
```

14. Checklist Verifica Rapida

Strumento	Comando di verifica	Output atteso
Docker	<code>docker --version</code>	Docker version X.X
Nikto	<code>nikto -Version</code>	Nikto main 2.1.5
Semgrep	<code>semgrep --version</code>	1.161.0
Dep-Check	<code>dependency-check --version</code>	Dependency-Check X.X
GitLeaks	<code>gitleaks version</code>	gitleaks X.X.X
DVWA	<code>curl -s -o /dev/null -w "%{http_code}" http://localhost:8081</code>	302
ZAP daemon	<code>curl -s http://localhost:8080/JSON/core/view/version/?apikey=...</code>	<code>{"version": "2.17.0"}</code>
ASTF	<code>astf --version</code>	OWASP API Security... 1.0.0
WuppieFuzz	<code>wuppiefuzz version</code>	wuppiefuzz 1.2.0
Falco	<code>ps aux grep falco grep -v grep</code>	falco processo attivo
owasp-scan	<code>bash -n ~/owasp-scan/owasp-scan.sh && echo OK</code>	Syntax OK

15. config.env — Guida Completa alle Variabili

Il file `config.env` è l'unico file da modificare per configurare `owasp-scan.sh`. Tutte le variabili opzionali hanno un valore di default sensato — se lasciate vuote la fase corrispondente viene saltata automaticamente.

i INFO Le variabili **OBBLIGATORIE** sono solo 3: `TARGET_URL`, `ZAP_PORT` e `ZAP_API_KEY`. Tutto il resto è opzionale.

15.1 Progetto

Variabile	Descrizione	Esempio	Req.
<code>PROJECT_NAME</code>	Nome del progetto. Usato nel titolo del report HTML e nei log. Non influenza il comportamento dello script.	<code>"myapp"</code>	Opzionale

15.2 Target

`TARGET_URL` è l'unica obbligatoria.

Variabile	Descrizione	Esempio	Req.
<code>TARGET_URL</code>	URL completo dell'applicazione web con schema e porta. Usato da Nikto, ZAP spider e ZAP active scan. Deve essere raggiungibile dalla VPS.	<code>"http://172.17.0.1:8081"</code>	Obbligatorio

Variabile	Descrizione	Esempio	Req.
<code>TARGET_API_URL</code>	URL base delle API REST per ASTF e WuppieFuzz. Se vuoto usa <code>TARGET_URL</code> come fallback.	"http://172.17.0.1:8082/api"	Opzionale
<code>TARGET_IP</code>	IP del target senza porta. Usato da OpenVAS per creare il target di scan tramite API REST.	"51.75.196.150"	Per OpenVAS

15.3 Codice Sorgente

Se `SOURCE_DIR` è vuoto le fasi GitLeaks, Semgrep e Dependency-Check vengono saltate automaticamente.

Variabile	Descrizione	Esempio	Req.
<code>SOURCE_DIR</code>	Path assoluto alla directory con il codice sorgente. Montata come volume Docker da Semgrep, GitLeaks e Dependency-Check. Deve essere una directory git inizializzata per GitLeaks.	"/home/ubuntu/myapp"	Per SCA

15.4 Credenziali API

ATTENZIONE Non usare credenziali di produzione. Usare sempre utenti di test dedicati con permessi minimi.

Variabile	Descrizione	Esempio	Req.
<code>API_TOKEN</code>	Bearer token JWT/OAuth per autenticazione. Passato ad ASTF con <code>--token</code> . Ha priorità su <code>API_USERNAME</code> se entrambi sono configurati. Includere il prefisso 'Bearer'.	"Bearer eyJhbGci..."	Opzionale
<code>API_USERNAME</code>	Username per Basic Authentication. Usato con <code>API_PASSWORD</code> . Ignorato se <code>API_TOKEN</code> è configurato.	"admin"	Opzionale
<code>API_PASSWORD</code>	Password per Basic Authentication. Usato con <code>API_USERNAME</code> .	"password"	Opzionale

15.5 ZAP

ATTENZIONE ZAP deve essere in modalità daemon per essere usato dallo script. Verificare sempre prima di lanciare.

Variabile	Descrizione	Esempio	Req.
<code>ZAP_PORT</code>	Porta API REST di ZAP. Deve corrispondere alla porta del	"8080"	Obbligatorio

Variabile	Descrizione	Esempio	Req.
ZAP_API_KEY	docker run (-p 8080:8080). Default consigliato: 8080. API key configurata nel docker run con -config api.key=VALORE. Deve corrispondere esattamente.	"dallaradallara20262026"	Obbligatorio

15.6 Dependency-Check

Variabile	Descrizione	Esempio	Req.
NVD_API_KEY	API key gratuita NVD per scaricare il database CVE senza timeout. Registrarsi su nvd.nist.gov/developers/request-an-api-key .	"ef5e1703-..."	Per SCA
CVSS_THRESHOLD	Soglia CVSS (0-10) per segnalare findings. 7.0 = High+Critical. 9.0 = solo Critical.	"7.0"	Opzionale

15.7 ASTF

Variabile	Descrizione	Esempio	Req.
ASTF_TIMEOUT	Timeout scansione in minuti. Su target locali 5-10 min. Su target remoti con rate limiting usare 20-30. Se scade il report non viene generato.	"10"	Opzionale
ASTF_THREADS	Thread paralleli. Max 8 su VPS con 4 core. Su target pubblici usare 3 per evitare rate limiting.	"8"	Opzionale

15.8 WuppieFuzz

Variabile	Descrizione	Esempio	Req.
OPENAPI_SPEC	Path assoluto alla specifica OpenAPI. DEVE contenere il campo 'servers'. Se vuoto WuppieFuzz viene saltato.	"/home/ubuntu/openapi.json"	Per WuppieFuzz
WUPPIE_TIMEOUT	Timeout fuzzing in secondi. 120s per test rapidi, 300s+ per fuzzing approfondito.	"120"	Opzionale
WUPPIE_AUTH_FILE	Path a file YAML con configurazione autenticazione. Supporta bearer, oauth, basic.	"/home/ubuntu/auth.yaml"	Opzionale

Formato file auth.yaml per Bearer:

```
mode: bearer
```

```
configuration:
  url: http://TARGET/login
  username: admin
  password: password
```

Formato file auth.yaml per OAuth:

```
mode: oauth
configuration:
  access_url: http://TARGET/oauth/token
  refresh_url: http://TARGET/oauth/refresh
  username: admin
  password: password
```

15.9 OpenVAS

i INFO OpenVAS viene avviato in background. Il report va recuperato manualmente dall'interfaccia Greenbone.

Variabile	Descrizione	Esempio	Req.
<code>OPENVAS_HOST</code>	IP o hostname della macchina Greenbone Community Edition. Se vuoto la fase OpenVAS viene saltata.	"192.168.1.100"	Per OpenVAS
<code>OPENVAS_PORT</code>	Porta Greenbone Web Interface. Default 9392.	"9392"	Opzionale
<code>OPENVAS_USER</code>	Username Greenbone Web Interface.	"admin"	Per OpenVAS
<code>OPENVAS_PASS</code>	Password Greenbone Web Interface.	"password"	Per OpenVAS

15.10 Report

Variabile	Descrizione	Esempio	Req.
<code>AUTO_SERVE</code>	Se 'true' avvia server HTTP Python alla fine della pipeline. Usare 'false' per nohup/CI-CD.	"true"	Opzionale
<code>SERVE_PORT</code>	Porta server HTTP report. Verificare che non sia in uso. Report su <code>http://IP:PORTA/index.html</code>	"9999"	Opzionale

15.11 Esempio config.env completo

```
PROJECT_NAME="myapp-backend"
TARGET_URL="http://staging.example.com:8080"
TARGET_API_URL="http://staging.example.com:8080/api/v1"
TARGET_IP="10.0.1.50"
SOURCE_DIR="/home/ubuntu/myapp"
API_TOKEN="Bearer eyJhbGciOiJIUzI1NiJ9..."
ZAP_PORT="8080"
ZAP_API_KEY="dallaradallara20262026"
NVD_API_KEY="ef5e1703-c84b-4df3-902b-15926c8d7ff6"
CVSS_THRESHOLD="7.0"
ASTF_TIMEOUT="10"
ASTF_THREADS="8"
OPENAPI_SPEC="/home/ubuntu/myapp/openapi.json"
WUPPIE_TIMEOUT="120"
WUPPIE_AUTH_FILE="/home/ubuntu/wuppie-auth.yaml"
OPENVAS_HOST="192.168.1.100"
```

```

OPENVAS_PORT="9392"
OPENVAS_USER="admin"
OPENVAS_PASS="password"
AUTO_SERVE="false"
SERVE_PORT="9999"

```

15.12 Riepilogo tutte le variabili

Variabile	Fase che usa	Default	Obbligatoria
PROJECT_NAME	Report	security-scan	No
TARGET_URL	Nikto, ZAP	—	Sì
TARGET_API_URL	ASTF, WuppieFuzz	= TARGET_URL	No
TARGET_IP	OpenVAS	—	No
SOURCE_DIR	Semgrep, GitLeaks, Dep-Check	—	No
API_TOKEN	ASTF	—	No
API_USERNAME	ASTF	—	No
API_PASSWORD	ASTF	—	No
ZAP_PORT	ZAP	—	Sì
ZAP_API_KEY	ZAP	—	Sì
NVD_API_KEY	Dependency-Check	—	No
CVSS_THRESHOLD	Dependency-Check	7.0	No
ASTF_TIMEOUT	ASTF	10 min	No
ASTF_THREADS	ASTF	8	No
OPENAPI_SPEC	WuppieFuzz	—	No
WUPPIE_TIMEOUT	WuppieFuzz	120s	No
WUPPIE_AUTH_FILE	WuppieFuzz	—	No
OPENVAS_HOST	OpenVAS	—	No
OPENVAS_PORT	OpenVAS	9392	No
OPENVAS_USER	OpenVAS	—	No
OPENVAS_PASS	OpenVAS	—	No
AUTO_SERVE	Report	false	No
SERVE_PORT	Report	9999	No